Crypto Vibeness

Découvrir le monde merveilleux de la crypto grâce au vibe coding

Introduction

La crypto (-graphie) est un domaine absolument passionnant… plus encore que **les** crypto (-monnaies) !

Elle se compose en gros de trois branches :

1. **les hash** (md5, sha256…) : un outil essentiel que l'on retrouve dans l'indexation de données, toutes les blockchains, la gestion de mots de passe, git… et même les RIB !
2. **la crypto symétrique :** utilisée dans l'intégralité des conversations numériques sécurisées (HTTPS, les VPNS, les réseaux WiFi protégés, les

emails d'un même provider, Signal…), le chiffrement de backups (iCloud, etc.), les paiements électroniques…

3. **la crypto asymétrique** : c'est la plus grande invention cryptographique du XXe siècle. Elle permet de communiquer de façon sécurisée avec n'importe qui **sans avoir au préalable de secret commun**. On l'utilise pour les échanges initiaux de clé pour le trafic HTTPS, les signatures et certificats numériques, les connexions SSH… plus complexe, elle nécessite une paire "clé publique/clé privée".

Le but du projet

Le but de ce projet est de **vous faire vibe-coder un système de chat sécurisé en Python**. Il utilisera progressivement ces trois branches. Vous n'aurez **rien à coder vous-mêmes** (c'est presque impossible en si peu de temps), mais vous aurez besoin de **comprendre les concepts essentiels de la crypto** pour vérifier l'output de votre agent !

—

Bien prompté, un LLM moderne peut vous sortir une implémentation d'un algo de chiffrement en quelques secondes. Ceci ne nous intéresse pas vraiment. Ce qui nous intéresse plutôt, c'est de **vous faire construire un système cryptographique complet**, prompt après prompt, en validant régulièrement votre avancement.

Pour ce faire, vous allez devoir utiliser un agent IA en ligne de commandes (Copilot CLI, Claude Code, Codex, Gemini CLI…). Vous devrez utiliser toutes les **bonnes pratiques associées aux agents** : context engineering, fichier Markdown de configuration, etc.

La clé : **à chaque étape, vous devrez valider vous-mêmes que le projet est valide et répond aux exigences**. Seulement à ce moment-là pourrez-vous passer à l'étape supérieure. N'essayez pas de sauter une étape, et **pensez à commit à chaque fois que votre travail progresse !**

Et surtout… utilisez les LLMs comme tuteur pour vous expliquer les concepts ! (cf. le Study Mode de ChatGPT)

Votre système cryptographique

Durant votre quête d'un système sécurisé, vous allez rencontrer des personnes mal intentionnées. **Trois hackers successifs vont en effet mettre votre système à rude épreuve** : un hacker marseillais, puis un hacker russe, et enfin un hacker de la NSA. **Votre objectif sera de sécuriser votre système contre ces adversaires…** si vous le pouvez !

Le but n'est pas de vous faire implémenter vous-mêmes les primitives cryptographiques dont vous aurez besoin, mais de prompter intelligemment, avec le bon contexte, pour arriver exactement à votre objectif.

Avertissement sur la crypto

La cryptographie est un domaine ardu et subtil, qu'on a tôt fait de mal utiliser. Dans la vraie vie, **réimplémenter soi-même des mécanismes cryptographiques est presque toujours une erreur**.

Le but de ce projet n'est pas de vous faire concevoir un système réellement sécurisé, ni de produire du code utilisable en production. L'objectif est de **manipuler concrètement les concepts fondamentaux de la cryptographie** : hash, dérivation de clés, chiffrement symétrique et asymétrique, échange de clés, signatures, chiffrement de bout en bout…

Les systèmes proposés dans ce sujet sont volontairement simplifiés, parfois naïfs, et peuvent être cryptographiquement faible et donc dangereux dans un contexte réel. Cela est assumé.

SI vous travaillez sur un projet où la sécurité est essentielle, ne reproduisez pas ces systèmes sans réfléchir, **utilisez toujours des bibliothèques éprouvées**. Ou comme disent les pros : "Don't roll your own crypto" (sauf si c'est pour apprendre, bien sûr !)

Indications :

- Vous êtes invité.e.s à lire le sujet de la semaine entière avant de vous lancer
- Vous devez utiliser un **agent IA**, pas un vulgaire chatbot
- Réfléchissez à ce que vous allez mettre dans votre fichier `...` `.md`. Chaque élément de contexte est important.
- Vous pouvez communiquer avec votre agent en français, mais **le code doit être intégralement en anglais.**

Jour 1 - 1ère partie : YOLO

« From a security perspective, if you're connected, you're screwed. » — **Daniel "DJB" J. Bernstein**

Pour commencer, vous souhaitez juste avoir un système de chat multi-utilisateurs type IRC, sans authentification, ni chiffrement. Vous devez commencer avec deux fichiers, `server.py` et `client.py`.

Fonctionnalités attendues :

- Vous avez un **serveur de chat** qui écoute sur un port passé en paramètre (ou un port par défaut si pas de paramètres, défini dans une variable de configuration). Il peut gérer **plusieurs clients simultanément**. Ceux-ci doivent passer le même port (ou bien se connecter au port par défaut).

- **Chaque client choisit son username** au début du chat. Le serveur s'assure qu'un même username ne peut pas se connecter plusieurs fois en même temps.
- Votre serveur est doté d'un **système de rooms**, dont certaines sont protégées par mot de passe. Un utilisateur peut créer et rejoindre des rooms. Par défaut, tout le monde est dans la room `general`.
- Chaque client **ne voit que les messages de la room où il se trouve**, associés à un timestamp.
- Une couleur est affectée automatiquement aux clients par le serveur, de façon déterministe (vous pouvez utiliser comme base le combo "IP+port", le username…). Elle est vue de la même façon chez tous les clients et pour toute la durée du chat.
- Le serveur doit **produire des logs des différents événements** associés à chaque client. Ces logs sont exportés dans un fichier type `log_2026-04-14_10-23-45.txt`
- Les rooms protégées par mot de passe s'affichent différemment dans la console (à vous de trouver un élément distinctif…).

Une fois ceci obtenu, assurez-vous que tout fonctionne. Chaque fonctionnalité évoquée ci-dessus doit être **testée et validée**. Ne passez pas à la suite si vous n'avez pas ce fonctionnement.

Jour 1 - 2e partie : Ô temps ! ...tification

« Rule 1 of cryptanalysis: check for plaintext. » — **Robert Morris**

Impossible de savoir qui se cache derrière ces pseudos ! Vos utilisateurs profitent de l'anonymat pour troller en disant que Windows est un OS sécurisé et bien conçu. Indigné, vous décidez de mettre en place un **système d'authentification**.

Features attendues :

- Les clients doivent **s'authentifier** par un mot de passe pour accéder au chat. Les clients non-authentifiés ne reçoivent pas les messages.
- Cette authentification se fait **avant** l'accès au chat, la création de rooms, etc.
- Soit le user est reconnu par le serveur dans sa table de mots de passe, soit il est créé à sa première connexion, avec un mot de passe choisi puis **confirmé** par lui.
- Vous devez implémenter au moins **3 règles** pour le choix du mot de passe. Si ces règles sont respectées par le user, vous enregistrez ses identifiants et vous retournez **un indicateur de la force du mot de passe**, fondé par exemple sur son **entropie**.
- Tous les mots de passe sont conservés hashés : **interdiction de les stocker en clair**.

- Inconscient des dangers du monde qui nous entourent, vous choisissez l'algorithme **md5** pour le hash (on ne vous demande pas d'implémenter vous-même cet algo).
- La table de mots de passe est maintenue dans un fichier "`this_is_safe.txt`" sur le serveur. Elle contient une ligne par utilisateur et chaque ligne correspond au format : `username:hashed_password`. Le hash est encodé en **base64** (et pas en hexadécimal).
- Vos règles de mots de passe sont conservées dans un fichier à part, dans le format de votre choix. Elles sont modifiables et lues à chaque lancement du serveur.
- La vérification du mot de passe se fait en temps constant.

Une fois ceci obtenu, assurez-vous que tout fonctionne une fois de plus. Chaque fonctionnalité évoquée ci-dessus doit être **testée et validée**. Ne passez pas à la suite si vous n'avez pas ce fonctionnement.

Jour 2 - 1ère partie : le hacker marseillais

« *When in doubt, use brute force.* » — **Ken Thompson** (Turing Award 1983)

On vous a prévenu qu'un hacker marseillais rôdait dans les couloirs. Peu sophistiqué, il se contente d'attendre votre départ au toilettes pour récupérer le fichier de mots de passe sur une clé USB.

Pour mieux vous en protéger, vous décidez de vous mettre à sa place et d'essayer de **casser les mots de passe de vos utilisateurs** (= retrouver le texte qui a généré le hash). Vous utilisez pour cela le programme **hashcat** (évitez de le recompiler from scratch ! Utilisez apt-get, brew, une image docker…).

Vous découvrez aussi que ce mystérieux hacker marseillais a laissé en partant un fichier sur votre serveur : "`md5_yolo.txt`". Vous investiguez :

```
?> cat md5_yolo.txt
35b95f7c0f63631c453220fb2a86f218
```

Features attendues :

- Vous arrivez à **déterminer le message laissé par le hacker** en lançant hashcat avec le masque "?u?u?l?l?u?u?s" et le mode "brute force". Le message du hacker et la commande pour le casser sont stockés dans un fichier `md5_decrypted.txt`.
- Dans la foulée, vous apprenez à **casser tous les mots de passe de vos utilisateurs** de 5 caractères ou moins, en un temps raisonnable.
- Vous remplacez votre vilain md5 par une fonction de hash moderne et sécurisée (par exemple des algos récents finalistes du **NIST,** ou n'importe quel cipher inventé par DJB), qui doit avoir un **facteur de coût**. Ce facteur devra figurer dans les infos stockées dans la table de mots de passe.

- Vous salez vos mots de passe avant de les hasher. Votre sel devra avoir au moins 96 bits et être différent pour chaque mdp. Il sera stocké à gauche du digest, lui aussi en **base64**.
- Vous devez avoir une séparation visuelle entre user, algo, salt et mdp, par exemple le caractère ":"

`jeanmichel:bcrypt:12:xt8pd3xi:ab4%x…` (algo "bcrypt", facteur de coût de 12, le salt est `xt8pd3xi`, et le digest du hash commence par `ab4%x`)

Jour 2 - 2è partie : le hacker russe

« There are only a few hard problems in cryptography — one of them is correctly implementing cryptography. » — **Adi Shamir** (Turing Award 2002)

Le site Marsactu vous apprend qu'un hacker russe est parvenu à enregistrer tout le trafic de l'école 😱Vous décidez donc d'**implémenter le chiffrement symétrique de toutes les conversations** !

Features attendues :

- Vous **générez une clé de chiffrement à la création de chaque compte** à partir d'un secret saisi par l'utilisateur, en utilisant une fonction de dérivation de clé. Assurez-vous d'utiliser un sel dédié par utilisateur, avec les mêmes contraintes qu'à la partie 2.
- Côté serveur, cette clé est conservée, avec le sel, dans un fichier "`user_keys_do_not_steal_plz.txt`", sur le même format que les mots de passe. Côté client, où vous voulez ("./users/babar/key.txt" par exemple…).

- Cette clé doit avoir une taille d'**au moins 128 bits.**
- Chaque utilisateur utilise ensuite cette clé pour **chiffrer ses messages et les envoyer au serveur**. Le nouveau client peut ensuite communiquer avec tous les utilisateurs connectés.
- Pour le cipher, vous pouvez au choix implémenter le TEA ou piocher dans les ciphers de libsodium/openssl et binder depuis votre programme. Dans tous les cas, vous devrez utiliser du **chiffrement par bloc**.

Jour 3 - 1ère partie : Do you have anything to hide?

« Anyone, from the most clueless amateur to the best cryptographer, can create an algorithm that he himself can't break. It's not even hard. What is hard is creating an algorithm that no one else can break, even after years of analysis. » — **Bruce Schneier**

Il est très pénible d'échanger les clés de chiffrement, et vous réalisez que **vos utilisateurs finissent par se les échanger via des canaux non-sécurisés**… ce qui annule complètement l'intérêt du chiffrement ! Et votre cousine (qui travaille justement à la NSA) vous apprend que l'agence s'intéresse de près à votre petit groupe de cryptographes amateurs. Il est temps de muscler votre jeu.

Pour éliminer le besoin de s'échanger des clés au préalable, **vous décidez d'implémenter un chiffrement asymétrique pour votre système** : vous utilisez un système de clé privée/clé publique pour échanger un secret qui vous servira ensuite à un chiffrement symétrique, beaucoup plus rapide.

Features attendues :

- Le client génère une paire de clés avant la connexion (ou bien réutilise une paire existante).
- Vous utilisez le chiffrement asymétrique pour vous échanger une clé de chiffrement symétrique.
- Vous utilisez ensuite cette clé pour communiquer, sur le modèle de la partie 4 (encapsulation de clé).
- Vous n'avez plus de fichier "`user_keys_do_not_steal_plz.txt`" côté serveur. Côté client, vos clés sont stockées dans deux fichiers de même nom qui finissent par "`.priv`" et "`.pub`".

Jour 3 - 2e partie : des bouts de bout en bout

« If you are designing cryptosystems, you've got to think about long-term applications. You've got to try to figure out how to build something that is secure against technology in the next century that you cannot even imagine. » — **Whitfield Diffie** (Turing Award 2015)

Vous savez de source sûre que la NSA a noyauté votre petit groupe… un de vos associés leur enverrait même les logs du serveur en douce. Vous ne pouvez même plus faire confiance au serveur ! Une seule solution :

implémenter une solution pour que le serveur lui-même ne connaisse pas le contenu des échanges. C'est ce qu'on appelle le chiffrement de bout en bout (ou E2EE). **Vous ne devrez l'implémenter que pour les échanges 1-1**. Les messages devront être signés et vérifiés par chaque partie.

Note : le serveur est honnête-mais-curieux : il route fidèlement les messages mais essaie de lire leur contenu. Il ne modifie ni ne supprime de messages (c'est le modèle Signal).

Étape 1 : distribution des clés publiques via le serveur

Quand un client se connecte, il envoie sa clé publique au serveur. Le serveur maintient un annuaire `{username: public_key}` et le distribue aux autres clients. Chaque client stocke localement les clés publiques des gens avec qui il a déjà communiqué. **Validation :** un client peut afficher la clé publique d'un autre client sans que le serveur ait accès aux clés privées.

Étape 2 : établissement d'une clé de session par paire de clients

Quand Alice veut parler à Bob, elle génère une clé symétrique de session, la chiffre avec la clé publique de Bob, et l'envoie via le serveur. Bob la déchiffre avec sa clé privée. Les deux ont maintenant un secret partagé que le serveur ne connaît pas. **Validation :** loguer côté serveur le message chiffré qui transite, montrer qu'il est illisible.

Étape 3 : chiffrement des messages avec la clé de session

Tous les messages entre Alice et Bob sont désormais chiffrés en symétrique avec cette clé de session (même mécanisme que le Jour 2 partie 2, réutilisation directe du code). Le serveur ne fait que relayer des blobs opaques. **Validation :** les logs serveur ne contiennent que du chiffré.

Étape 4 : signature des messages

Chaque message est signé avec la clé privée de l'expéditeur avant envoi. Le destinataire vérifie la signature avec la clé publique de l'expéditeur. Si la vérification échoue, le message est rejeté et le client est averti. **Validation :** altérer manuellement un octet dans un message en transit (par exemple avec un petit proxy), vérifier que le destinataire le rejette.

Compétences visées

- Prompt engineering
- Cryptographie
- Réseau

Rendu

➔ Votre travail sera évalué en soutenance avec un support et une revue de code.

Base de connaissances

- le programme hashcat
- Le fabuleux Crypto 101 qui contient un cours essentiel sur les primitives cryptographiques
- how secure is my password?
- le jeu du password game pour vous donner des idées de règles de mots de passe… sinon les règles de l'ANSSI c'est bien aussi.
- sur l'entropie des mots de passe, l'inévitable xkcd.
- l'annonce de la première collision SHA-1.
- les bons réflexes en crypto par Latacora
- le system prompt de l'ancien "study mode" de ChatGPT, que vous pouvez recopier dans une conversation